

Number System

09 June 2026 05:08

How are numeric data items actually stored in the computer memory?

How much space(memory locations) is allocated for each type of data?

- byte, short, int, long, float, double, char, boolean.

How are characters and strings are stored in memory?

Number System:: The basics

We are accustomed to using the so called **decimal number system**.

- Ten digits:: 0,1,2,3,4,5,6,7,8,9
- Every digit position has a weight which is a power of 10.
- Base or radix is 10

Example:

$$234 = 2 * 10^2 + 3 * 10^1 + 4 * 10^0$$

$$250.67 = 2 * 10^2 + 5 * 10^1 + 0 * 10^0 + 6 * 10^{-1} + 7 * 10^{-2}$$

Binary Number System

Two digits:

- 0 and 1
- Every digit position has a weight which is a power of 2.
- Base or radix is 2

Example:

$$110 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0$$

$$110.01 = 1 * 2^2 + 1 * 2^1 + 0 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2}$$

Binary to decimal conversion

Each digit position of a binary number has a weight.

- Some power of two

A binary number:

$$B = b_{n-1}b_{n-2}...b_1b_0b_{-1}b_{-2}...b_{-m}$$

Corresponding value in decimal:

$$D = \sum_{i=-m}^{n-1} b_i \cdot 2^i$$

$$\sum_{i=-5}^{3-1=2} b_i \cdot 2^i$$

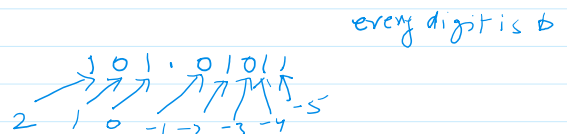
n = number of digits in integer part

Examples:

$$\begin{aligned} 101011_2 &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 32 + 0 + 8 + 0 + 2 + 1 \\ &= 43_{10} \end{aligned}$$

$$0.0101_2 = 0 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}$$

every digit is b



$$\begin{array}{l} 2^{10} \cdot 10^{-2} \\ \underline{101.11} = 5 + 1 \times 2^{-1} + 1 \times 2^{-2} \\ 5 + 0.50 + 0.25 = 5.75_{10} \end{array}$$

```

bs = '1101'
print(int(bs,2)) ← binary to int
13
print(oct(13)) ← int to octal
0o15
print(hex(13)) ← int to hex
0xd
print(bin(13)) ← int to binary
0b1101

```

Java

```
public class Coverersion
```

```
{
```

```
    public static void convertIntegerToBinary(int v){
```

```
        System.out.print("\nBinary equivalent of " + v + " is: " +
```

```
        Integer.toBinaryString(v));
```

```
        System.out.print("\n?: "+Integer.toUnsignedString(110, 2));
```

```
        System.out.print("\n ??: "+Integer.toString(1101, 16));
```

```
    }
```

```
}
```

int is converted to binary
(1101110)₂

Methods available in Java for conversion:

```

String toBinaryString(int)
String toHexString(int)
String toOctalString(int)
String toString(int)
String toString(int, int)
long toUnsignedLong(int)
String toUnsignedString(int)
String toUnsignedString(int, int)

```

Integer.to

Hexadecimal Number System

A compact way of representing binary numbers

16 different symbols(radix = 16)

16 = 2⁴ ← bits

0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	A (10)
1	0	1	1	B (11)

1	0	0	1	A (10)
1	0	1	1	B (11)
1	1	0	0	C (12)
1	1	0	1	D (13)
1	1	1	0	E (14)
1	1	1	1	F (15)

Binary to Hexadecimal Conversion

For the integer part:

- Scan the binary number from right to left.
- Translate each group of four bits into the corresponding hexadecimal digit.
 - Add leading zeros if necessary.

For the fractional part:

- Scan the binary number from left to right.
- Translate each group of four bits into the corresponding hexadecimal digit.
 - Add trailing zeros if necessary.

For example:-

$$\text{101011101.01101110}_2 = (2ed.67)_{16}$$

$$(0.10001000)_2 = (0.84)_{16}$$

8	4	2	1	0 to 15
1	1	0	1	
1	1	1	0	
1	0	0	0	
0	1	0	1	
1	0	1	0	
1	1	0	1	
0	0	1	1	
0	0	1	0	

Hexadecimal to binary Conversion

Translate every hexadecimal digit into its 4 bit binary equivalent.

$$(3A5)_{16} = (1110100101)_2$$

$$(12.3D)_{16} = (10010.0011101)_2$$

Octal Number System

A compact way of representing binary numbers

8 different symbols (radix or base = 8)

0	0	0	→	0
0	0	1	→	1
0	1	0	→	2
0	1	1	→	3
1	0	0	→	4
1	0	1	→	5
1	1	0	→	6
1	1	1	→	7

Binary to Octal Conversion

For the integer part:

- Scan the binary number from right to left
- Translate each group of three bits into a corresponding octal digit
 - Add leading zeros if necessary

For the fractional part:

- Scan the binary number from left to right
- Translate each group of three bits into the corresponding octal digit
 - Add trailing zeros if necessary

For example:

$$(1010101)_2 = (525)_8$$

$$(0.01010110)_2 = (0.256)_8$$

$$(100101.011010)_2 = (15.32)_8$$

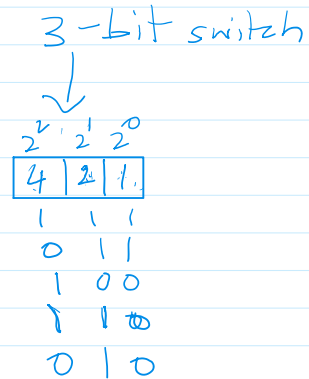
bit → switch $\begin{cases} \text{ON} \rightarrow 1 \\ \text{OFF} \rightarrow 0 \end{cases}$

Octal to binary Conversion

Translate every octal digit into its 3 bit binary equivalent.

$$(437)_8 = (100\ 011\ 111)_2$$

$$(53.26)_8 = (101011.010110)_2$$



Decimal to octal

$$(983)_{10} = (?)_8$$

8	983	
8	122	-7 ↑
8	15	-2
8	1	-7
	0	-1

$$(983)_{10} = (1727)_8$$

$$(8)_{10} = (?)_8$$

8	8	
8	1	-0 ↑
	0	-1

$$(8)_{10} = (10)_8$$

$$1/2048 = 0.0005$$

$$1/64 = 0.0156$$

$$(201.53)_{10} = (?)_8$$

For integer part-

8	201	
8	25	1 ↑
	1	

	53 × 8
4	24 × 8
1	92 × 8
7	02 × 8

$$(311.4172)_{10} = (?)_8$$

$$= 3 \times 8^2 + 1 \times 8^1 + 1 \times 8^0 + 4 \times 8^{-1} + 1 \times 8^{-2} + 7 \times 8^{-3} + 2 \times 8^{-4}$$

$$= 192 + 8 + 1 + \frac{4}{8} + \frac{1}{64} + \frac{7}{512} + \frac{2}{2048}$$

$$\begin{array}{r} 8 \overline{) 201} \\ 8 \overline{) 25} \\ 8 \overline{) 3} \\ \hline 0 \end{array} \quad \begin{array}{l} 1 \uparrow \\ 1 \uparrow \\ 3 \end{array}$$

$$(201.53)_{10} \approx (323.4172)_8$$

$$\begin{array}{r} 4 \quad 24 \times 8 \\ 1 \quad 92 \times 8 \\ 7 \quad 36 \times 8 \\ \hline 2 \quad 88 \end{array}$$

$$\begin{array}{r} 82 \quad 64 \quad 512 \quad 4096 \\ \hline 201 + .5 + 0.0156 + 0.0137 + 0.0005 \\ \hline 201.5298 \end{array}$$

$$0.0156 + 0.0137 + 0.0005 + 0.5 = 0.5298$$

Octal to decimal

Hexadecimal to decimal

$$(ABC)_{16} = (?)_{10}$$

$$101010111000$$

$$\begin{array}{r} 8 \quad 4 \quad 2 \quad 1 \\ \hline 1 \quad 1 \quad 0 \quad 0 \\ 1 \quad 0 \quad 1 \quad 1 \\ \hline 1 \quad 0 \quad 1 \quad 0 \end{array}$$

Hex to bin $\rightarrow$ dec

$$\begin{array}{c} 2048 \quad 1024 \quad 512 \quad 256 \quad 128 \quad 64 \quad 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1 \\ \hline 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \end{array}$$

$$2048 + 512 + 128 + 32 + 16 + 8 + 4 = 2748$$

$$2048 + 512 + 128 + 32 + 16 + 8 + 4 = 2748$$

Unsigned binary Numbers

A n-bit binary number

$$B = b_{n-1} b_{n-2} b_{n-3} \dots b_2 b_1 b_0$$

2^n distinct combinations are possible; 0 to $2^n - 1$

For example, for $n=3$, there are 8 distinct combinations.

$$000, 001, 011, 100, 101, 110, 111$$

Range of numbers that can be represented.

$$\begin{array}{ll} n=8 & 0 \text{ to } 2^8 - 1 (255) \\ n=16 & 0 \text{ to } 2^{16} - 1 (65535) \\ n=32 & 0 \text{ to } 2^{32} - 1 (4294967295) \end{array}$$

$$n=8$$

$$\begin{array}{c} 128 \quad 64 \quad 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1 \\ \hline 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \\ 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \end{array}$$

$$\rightarrow 0$$

$$\rightarrow 255$$

$$\text{unsigned integer} = \boxed{0 \text{ to } 2^n - 1} \quad \because n = \text{number of bits}$$

Signed Integer representation

Many of the numerical data items that are used in programs are signed (positive or negative).

Question:: How to represent sign?

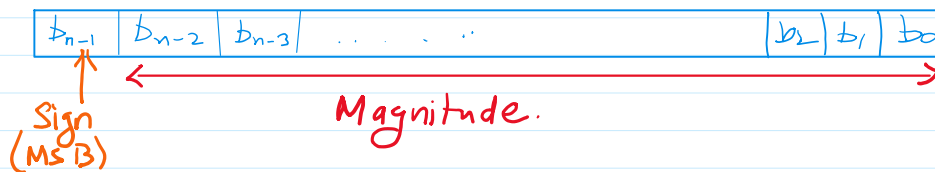
Three possible approaches:

- i. Sign-magnitude representation
- ii. One's complement representation
- iii. Two's complement representation

i. Sign-magnitude representation

For an N - bit representation

- The most significant bit (MSB) indicates sign
 - 0 -> positive
 - 1 -> negative
- The remaining (N - 1) bits represents magnitude



$$n = 8$$

Range of the number that can be represented:

$$\text{Maximum} :: + (2^{n-1} - 1)$$

$$\text{Minimum} :: - (2^{n-1} - 1)$$

$$\text{max} \Rightarrow 2^{8-1} - 1 = 2^7 - 1 = 127$$

$$\text{min} \Rightarrow -1(2^{8-1} - 1) = -1(2^7 - 1) = -127$$

7	6	5	4	3	2	1	0
-128	64	32	16	8	4	2	1
0	1	1	1	1	1	1	1
→ 1.	0	0	0	0	0	0	0
→ 0	0	0	0	0	0	0	0

← +127
- 0
+ 0

A problem:

Two different representation of zeros:

+0 -> 0 0000...0

-0 -> 1 0000...0

One's Complement Representation

Basic idea:

- Positive numbers are represented exactly as in sign-magnitude form.
- Negative numbers are represented in 1's complement form.

How to compute 1's complement of a number?

- Complement every bit of the number (1->0 and 0->1)
- MSB will indicate the sign of the number
 - 0 -> positive
 - 1 -> negative

- 0 -> positive
- 1 -> negative

Example :: $n = 4$

0 0 0 0 -> +0
 0 0 0 1 -> +1
 0 0 1 0 -> +2
 0 0 1 1 -> +3
 0 1 0 0 -> +4
 0 1 0 1 -> +5
 0 1 1 0 -> +6
 0 1 1 1 -> +7

1's complement → why?

MSB →

1	0	0	0	-> -7
1	0	0	1	-> -6
1	0	1	0	-> -5
1	0	1	1	-> -4
1	1	0	0	-> -3
1	1	0	1	-> -2
1	1	1	0	-> -1
1	1	1	1	-> -0

0 0 0 0
 1. 1 1 1 1
 + 1
 1 0 0 0

Range of the number that can be represented:

Maximum :: $+(2^{n-1} - 1)$

Minimum :: $-(2^{n-1} - 1)$

A problem:

Two different representation of zero:

+0 -> 0 0 0 0 ... 0

-0 -> 1 1 1 1 ... 1

Advantages of 1's complement representation

- Subtraction can be done using addition
- Leads to substantial saving in circuitry

Subtraction using Addition:: 1's Complement

How to compute $A - B$?

- Compute the 1's complement of B(say, B1)
- Compute $R = A + B1$
- If the carry obtained after addition is '1'
 - Add the carry back to R (called end-around carry)
 - That is $R = R + 1$
 - The result is a positive number
- Else
 - The result is negative and is in 1's complement form.

Algorithm.

Example:: $n = 4$ (bits)

6 - 2 using 1's complement

6 → 0 1 1 0
 2 → 0 0 1 0
 2' → 1 1 0 1

6 0 1 1 0
 2' 1 1 0 1
 1 0 0 1 1
 + 1
 0 1 0 0 (+4)

Example 2 :: 3 - 5

3 → 0011
5 → 0101
5' → 1010

0011
1010
1101

No carry
it means result is -ve.
(-2) 0010 = 2

Example: $-4 - 3 = -7$

$n = 4$ (bits)

4 in binary = 0100
4' (complement) = 1011

3 in binary = 0011
3' (complement) = 1100

1011
+ 1100
① 0111 ←
+1
1000 ← (-7)

What is 1011 = 11 in decimal, or (15-4)

What is 1100 = 12 in decimal, or (15-3)

1011 = 23 in decimal (15+15-4-3)

1000 ← (-7 in 1's comp)

↑ This is an exceptional case, where the 1's complement algorithm fails.
It fails when A is -ve and B is positive,

$$-A - (+B) = -A - B = -R$$

Subtract: $4 - 3$

4 binary 0100

3 binary: 0011
3' complement: 1100

0100
1100
10000
→ +1
0001

Subtract: $-4 + 3 = -1$

4 binary 0100
4' complement 1011

3 binary: 0011

$$\begin{array}{r} 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

$$\begin{array}{r} 10 \\ + 1 \\ \hline 11 \end{array}$$

4 bit

-8 | 4 | 2 | 1

Overflow? Example: 5+6

-7 to 7

$$5 \text{ binary} = 0101$$

$$6 \text{ binary} = 0110$$

$$\begin{array}{r} 0101 \\ 0110 \\ \hline 1011 \end{array} \rightarrow (-ve) \text{ in 1's complement.}$$

Overflows are handled separately

$$n = 8$$

7	6	5	4	3	2	1	0
-128	64	32	16	8	4	2	1

$$5 \text{ binary} = 0000 \ 0101$$

$$6 \text{ binary} = + 0000 \ 0110$$

$$0000 \ 1011 \rightarrow +11$$

To handle overflows we need increase number of bits.
Therefore, $N = n \times 2$ will be applied. [$\because n$ is current number of bits]

How to handle the problem of having two 0's?

Two's Complement Representation

Basic idea:

- Positive numbers are represented exactly as in sign-magnitude form.
- Negative numbers are represented in 2's complement form.

How to compute 2's complement of a number?

- Complement every bit of the number (1 $\rightarrow$ 0 and 0 $\rightarrow$ 1), and then add one to the resulting number.
- MSB will indicate the sign of the number
 - 0 $\rightarrow$ positive
 - 1 $\rightarrow$ negative

Two's complement

Range of numbers that can be represented:

Maximum :: $+(2^{n-1} - 1)$

Minimum :: $-(2^{n-1})$

Advantage:

- A unique representation of zero
- Subtraction can be done using addition
- Leads to substantial saving in circuitry

Almost all computers today use the two's complement representation for storing negative numbers.

Example: $n = 4$

Subtract $(6 - 3)$ using 2's complement

Binary of 6 = 0 1 1 0

Binary of 3 = 0 0 1 1

3's complement = 1 1 0 0

The 2's complement of 3 is = 1 1 0 0 + 1 = 1 1 0 1

Add the binary of 6 with 2's complement of 3:

0 1 1 0
1 1 0 1

✓ 0 0 1 1 (+3).

Subtract $(3 - 6)$ using 2's complement

Binary of 3 is = 0 0 1 1

Binary of 6 is = 0 1 1 0

6 one's complement = 1 0 0 1

6 two's complement = 1 0 0 1 + 1 = 1 0 1 0

0 0 1 1
1 0 1 0
— 1 1 0 1 (no carry) -ve number

↓
(-3)

↘ complement

0 0 1 0
+ 1
— 0 0 1 1 = 3

Subtract $(-3 - 6)$ using 2's complement

Binary of 3 = 0 0 1 1

Two's complement = 1 1 0 0 + 1 = 1 1 0 1

Binary of 6 = 0 1 1 0

Two's complement = 1 0 0 1 + 1 = 1 0 1 0

Add:

1 1 0 1
1 0 1 0
+ 0 1 1 1

↘ 2's complement sum

$\times \underline{0111}$ $\downarrow$ 2's complement form.
(-0111)
 $1000 + 1 = 1001 = \textcircled{9}$